jQuery - Event Handling

Any modern web application can't be imagined without an event associated with it. Events are the mechanism to build an interactive web page. jQuery is smart enough to handle any event generated on an HTML page. First let's try to understand what is an event.

What are jQuery Events?

A jQuery Event is the result of an action that can be detected by jQuery (JavaScript). When these events are triggered, you can then use a custom function to do pretty much whatever you want with the event. These custom functions are called **Event Handlers**.

The jQuery library provides methods to handle all the DOM events and make complete event handling considerably easier than what we have available in JavaScript.

Following are the examples of some common events –

- A mouse click
- A web page loading
- Taking mouse over an element
- Submitting an HTML form
- A keystroke on your keyboard, etc.

The following table lists some of the important DOM events.

Mouse Events	Keyboard Events	Form Events	Document Events
click	keypress	submit	load
dblclick	keydown	change	resize
hover	keyup	select	scroll
mousedown		blur	unload
mouseup		focusin	ready

This chapter coveres only few event methods and properties, For a complete reference of all the jQuery Event Methods and Properties, you can go to through jQuery Events Reference.

jQuery Event Binding Syntax

Consider a situation when you want to click a in an HTML document and then you want to perform some action against this click. To achieve this you will have to bind a jQuery **click** event with the element and then define an action against the click event.

Following is jQuery syntax to bind a click event with all the elements available in an HTML document:

```
$("div").click();
```

The next step is to define an action against the click event. Following is the syntax to define a function which will be executed when **click** event will be fired. This function is called **jQuery Event Handler**

```
$("div").click(function(){
   // jQuery code goes here
});
```

Following is another syntax to bind a click event with any of the DOM elements:

```
$("div").bind('click', function(){
   // jQuery code goes here
});
```

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

jQuery Event Examples

jQuery click Event

Following is an example to bind a **click** event with where an alert box is displayed whenever you click on any of the divs. Try to click the icon to run the following jQuery code:

</> Open Compiler

```
<!doctype html>
<html>
<head>
<title>The jQuery Example</title>
```

```
<script src="https://www.tutorialspoint.com/jquery/jquery-3.6.0.js"></script>
<script>
   $(document).ready(function() {
      $("div").click(function(){
         alert('Hi there!');
      });
   });
</script>
<style>
   div{ margin:10px;padding:12px; border:2px solid #666;
width:60px;cursor:pointer}
</style>
</head>
<body>
   <p>Click on any of the squares to see the result:</p>

   <div>One</div>
   <div>Two</div>
   <div>Three</div>
</body>
</html>
```

jQuery dblclick Event

Let's re-write the above code to bind a **dblclick** event with where an alert box is displayed whenever you double click on any of the divs.

</> Open Compiler

```
<!doctype html>
<html>
<head>
<title>The jQuery Example</title>
<script src="https://www.tutorialspoint.com/jquery/jquery-3.6.0.js"></script>
<script>
   $(document).ready(function() {
      $("div").dblclick(function(){
         alert('Hi there!');
      });
   });
</script>
```

```
<style>
   div{ margin:10px;padding:12px; border:2px solid #666;
width:60px;cursor:pointer}
</style>
</head>
<body>
   <p>Double click on any of the squares to see the result:</p>

   <div>One</div>
   <div>Two</div>
   <div>Three</div>
</body>
</html>
```

jQuery mouseenter Event

Following is an example to bind a **mouseenter** event with where an alert box is displayed whenever you bring cursor over any of the divs.

</>

Open Compiler

```
<!doctype html>
<html>
<head>
<title>The jQuery Example</title>
<script src="https://www.tutorialspoint.com/jquery/jquery-3.6.0.js"></script>
<script>
   $(document).ready(function() {
      $("div").mouseenter(function(){
         alert('Cursor is in!');
      });
   });
</script>
<style>
   div{ margin:10px;padding:12px; border:2px solid #666;
width:60px;cursor:pointer}
</style>
</head>
<body>
   <p>Bring cursor over any of the squares to see the result:</p>
```

```
   <div>One</div>
   <div>Two</div>
   <div>Three</div>
</body>
</html>
```

jQuery mouseleave Event

Following is an example to bind a **mouseleave** event with where an alert box is displayed whenever you take cursor out of the div.

</> Open Compiler

```
<!doctype html>
<html>
<head>
<title>The jQuery Example</title>
<script src="https://www.tutorialspoint.com/jquery/jquery-3.6.0.js"></script>
<script>
   $(document).ready(function() {
      $("div").mouseleave(function(){
         alert('Curosr is out!');
      });
   });
</script>
<style>
   div{ margin:10px;padding:12px; border:2px solid #666;
width:60px;cursor:pointer}
</style>
</head>
<body>
   <p>Take cursor out any of the squares to see the result:</p>

   <div>One</div>
   <div>Two</div>
   <div>Three</div>
</body>
</html>
```

jQuery mousedown Event

Following is an example to bind a **mousedown** event with where an alert box is displayed whenever the left, middle or right mouse button is pressed down over any of the divs.

```
<!doctype html>
<html>
<head>
<title>The jQuery Example</title>
<script src="https://www.tutorialspoint.com/jquery/jquery-3.6.0.js"></script>
<script>
   $(document).ready(function() {
      $("div").mousedown(function(){
         alert('Mouse button is down!');
      });
   });
</script>
<style>
   div{ margin:10px;padding:12px; border:2px solid #666;
width:60px;cursor:pointer}
</style>
</head>
<body>
   <p>Press mouse button down over any of the squares to see the result:</p>

   <div>One</div>
   <div>Two</div>
   <div>Three</div>
</body>
</html>
```

jQuery mouseup Event

Following is an example to bind a **mouseup** event with where an alert box is displayed whenever the left, middle or right mouse button is released over any of the divs.

```
<!doctype html>
<html>
<head>
<title>The jQuery Example</title>
<script src="https://www.tutorialspoint.com/jquery/jquery-3.6.0.js"></script>
<script>
   $(document).ready(function() {
      $("div").mouseup(function(){
         alert('Mouse button is released!');
      });
   });
</script>
<style>
   div{ margin:10px;padding:12px; border:2px solid #666;
width:60px;cursor:pointer}
</style>
</head>
<body>
   <p>Release mouse button over any of the squares to see the result:</p>

   <div>One</div>
   <div>Two</div>
   <div>Three</div>
</body>
</html>
```

jQuery Event Object

Whenever a jQuery event is fired, jQuery passes an **Event Object** to every event handler function.The event object provides various useful information about the event.

The event object is often unnecessary and the parameter is omitted, as sufficient context is usually available when the handler is bound to know exactly what needs to be done when the handler is triggered, however there are certain attributes which you would need to be accessed.

The following event properties/attributes are available and safe to access in a platform independent manner –

Property	Description

altKey	Set to true if the Alt key was pressed when the event was triggered, false if not. The Alt key is labeled Option on most Mac keyboards.
ctrlKey	Set to true if the Ctrl key was pressed when the event was triggered, false if not.
data	The value, if any, passed as the second parameter to the bind() command when the handler was established.
keyCode	For keyup and keydown events, this returns the key that was pressed.
metaKey	Set to true if the Meta key was pressed when the event was triggered, false if not. The Meta key is the Ctrl key on PCs and the Command key on Macs.
pageX	For mouse events, specifies the horizontal coordinate of the event relative from the page origin.
pageY	For mouse events, specifies the vertical coordinate of the event relative from the page origin.
relatedTarget	For some mouse events, identifies the element that the cursor left or entered when the event was triggered.
screenX	For mouse events, specifies the horizontal coordinate of the event relative from the screen origin.
screenY	For mouse events, specifies the vertical coordinate of the event relative from the screen origin.
shiftKey	Set to true if the Shift key was pressed when the event was triggered, false if not.
target	Identifies the element for which the event was triggered.
timeStamp	The timestamp (in milliseconds) when the event was created.
type	For all events, specifies the type of event that was triggered (for example, click).
which	For keyboard events, specifies the numeric code for the key that caused the event, and for mouse events, specifies which button was pressed (1 for left, 2 for middle, 3 for right).

Example

Following is an example to show how different square clicks give different coordinates.

```
<!doctype html>
<html>
<head>
<title>The jQuery Example</title>
<script src="https://www.tutorialspoint.com/jquery/jquery-3.6.0.js"></script>
<script>
   $(document).ready(function() {
      $("div").click(function(eventObj){
         alert('Event type is ' + eventObj.type);
         alert('pageX : ' + eventObj.pageX);
         alert('pageY : ' + eventObj.pageY);
         alert('Target : ' + eventObj.target.innerHTML);
      });
   });
</script>
<style>
   div{ margin:10px;padding:12px; border:2px solid #666;
width:60px;cursor:pointer}
</style>
</head>
<body>
   <p>Click on any of the squares to see the result:</p>

   <div>One</div>
   <div>Two</div>
   <div>Three</div>
</body>
</html>
```

this Keyword in Event Handler

Many times it becomes very easy to make use of **this** keyword inside an event handler. This keyword represents a DOM element which triggers the event.

Following example will show the content of the clicked :

```
<!doctype html>
<html>
<head>
<title>The jQuery Example</title>
<script src="https://www.tutorialspoint.com/jquery/jquery-3.6.0.js"></script>
<script>
   $(document).ready(function() {
      $("div").click(function(){
         alert($(this).text());
      });
   });
</script>
<style>
   div{ margin:10px;padding:12px; border:2px solid #666;
width:60px;cursor:pointer}
</style>
</head>
<body>
   <p>Click on any of the squares to see the result:</p>

   <div>One</div>
   <div>Two</div>
   <div>Three</div>
</body>
</html>
```

Removing Event Handlers

Typically, once an event handler is established, it remains in effect for the remainder of the life of the page. There may be a need when you would like to remove event handler.

jQuery provides the **unbind()** command to remove an exiting event handler. The syntax of unbind() is as follows –

```
selector.unbind(eventType, handler)

or

selector.unbind(eventType)
```

Following is the description of the parameters –

- **eventType** – A string containing a JavaScript event type, such as click or submit. Refer to the next section for a complete list of event types.
- **handler** – If provided, identifies the specific listener that's to be removed.

jQuery Events Reference

You can get a complete reference of all the jQuery Event Methods and Properties at the following page: jQuery Events Reference.